DS 210 Final Write Up

A. Project Overview

- **Goal**: The question I am answering is what are the natural groups of users that emerge based on age, swipe, behavior, interests, and demographic? I segmented artificial dating app users into behavioral cohorts by applying K-means clustering to their usage and profile features.

- **Dataset**: The input/dataset is dating_app_behavior_dataset_extended1.csv. This is a synthetic user-behavior table with 50,000 rows and 25 columns. The columns include categories such as age, gender, mutual_matches, interest_tags, and etc. To keep the runtimes reasonable, I only sampled 1000 records at runtime. This also means that everytime you run the program, each output will vary slightly.

B. Data Processing

- **Loading:** In data.rs, I used Rust's csv crate to stream each record into a HashMap<String,String>, preserving header to value mappings.

- **Sampling:** If the full dataset exceeds –sample (which default is set to 1000), the program randomly shuffles and truncates via rand::seq::SliceRandom.

- **Transformation:** If the full dataset exceeds –sample (which default is set to 1000), the program randomly shuffles and truncates via rand::seq::SliceRandom.

C. Code Structure

1. **Modules**

    - **main.rs**: Orchestrates CLI parsing, logging setup, data ingestion, clustering, and reporting the output. It's a top level orchestrator and holds the logic that ties together all the different modules.
    - **data.rs:** Takes the raw CSV data into memory and implements the sampling. It isolates I/O (input and output) and random-sampling. This means you can edit the source without touching the main, downstream code.
    - **features.rs:** In this, FeatureBuilder is defined, which inspects the user-attribute records to discover numeric fields, categorical domains, and interest tags. It then produced fixed length vectors of f64 element feature vectors. This module has all vector construction

logic (raw data to numeric, categorical, and tag encoding) so when we give the data to the clustering module, it only sees clean, numeric arrays.
- **clustering.rs**: This module implements a standalone k-means function that allows for partitioning the feature matrix in k clusters. By having this in its own module, it keeps the unsupervised learning algorithm separate from the data loading and I/O, so kmeans can be used on any Vec<Vec<f64>>.
- **analytics.rs**: This provides the reporting utilities cluster_sizes, which tallies labels, and write_assigments_csv, which exports results. Again, this separates downstream analytic from the core algorithm logic, which would allow for any easy extension of this dataset (i.e. adding silhouette coefficient).

2. **Key Functions & Types (Structs, Enums, Traits, etc)**

- load_records(path: &str) -> Result<Vec<HashMap<String,String>>> (data.rs)
  - Purpose: reads each row of the CSV into a HashMap of a header to a value. It preserves all 25 columns.
  - Input: the file path.
  - Output: A Vec<HashMap<String,String>>.
  - Logic: Uses csv::Reader to iterate through records, coone headers, and zip headers and values into each map.
- sample_records(records: &[HashMap<…>], n: usize) -> Vec<HashMap<…>> (data.rs)
  - Purpose: Randomly select at most n records to control dataset size for clustering.
  - Input: Full record slice and desired sample size.
  - Output: New vec of length of less than or equal to n.
  - Logic: Uses rand::seq::SliceRandom to shuffle in place, then truncates.
- FeatureBuilder::new(records: &[HashMap<…>]) -> Self (features.rs)
  - Purpose: scan the dataset once to collect a list of numeric field names, all unique categorical values per categorical column, all unique interest_tag values.
  - Input: all records.
  - Output: A builder struct containing numeric_fields: Vec<String>, categorical_values: HashMap<String,Vec<String>>, and tag_list: Vec<String>.
  - Logic: iterates through records to populate HashSets for each categorical field and interest_tags. It then converts sets to sorted Vecs for ordering.
- FeatureBuilder::vectorize(records: &[…]) -> Vec<Vec<f64>> (features.rs)
  - Purpose: convert each record into a single Vec<f64> by parsing numeric fields (or 0.0) in the order given by numeric_fields. Then does one-hot encoding on categorical fields by checking equality against each option. It then binary encodes each tag's presence.
  - Input: Slice of maps and builder's stored field lists.
  - Output: A matrix of shape (num_records) * (total_dimensions)
  - Logic: iterates through list of numeric column names, looks up string value, pushes resulting number onto vector. Returns 1.0 if the column matches the record's category, 0.0 is not. Now with N binary features, split the record's interest_tags on command and form a HashSet of present tags. For each tag in

the master list, emit 1.0 if the record has it, else 0.0, which will result in a Vec<f64>.

- kmeans(data: &[Vec<f64>], k: usize, max_iter: usize) -> (Vec<Vec<f64>>, Vec<usize>) (clustering.rs)
  - Purpose: cluster the provided feature matrix in k groups.
  - Inputs:
    1. Data: slice of n feature vectors
    2. k: number of clusters
    3. Max_iter: upper bound on the number of iterations.
  - Outputs:
    1. Centroids: a Vec of k centroid vectors
    2. Assignments: a vector of length n of cluster indices
  - Logic: This function initializes centroids at the first k points, assigns each point to its nearest centroid using square Euclidean distance, and recomputes each centroid as the mean of its assigned points. This process repeats until no assignments change or max_iter is reached.
- cluster_sizes(assignments: &[usize], k: usize) -> Vec<usize> (analytics.rs)
  - Purpose: counts how many records belong to each cluster.
  - Input: The cluster assignments and the value of k.
  - Output: A vector of length k of counts.
  - Logic: tallies how many points fell into each cluster by initializing a Vec<usize> of length k, of all zeroes. It then iterates over assignments and for each cluster ID c, does sizes[c] +=1. It then returns the sizes vector, so sizes[i] = number of records assigned to cluster i.
- write_assignments_csv(assignments: &[usize], path: &str) -> Result<(),_> (analytics.rs)
  - Purpose: persist row_index, cluster pairs for downstream reporting.
  - Input ;assignments and target file path.
  - Outputs: CSV file with 2 columns.
  - Logic: create a csv::Writer pointed at the given file path, write a header row (["row_index", "cluster"]), loop over assignments.iter().unermate(), and flush the writer to ensure all buffers hit disk. Return Ok(()), which gives a clean CSV.

3. **Main Workflow**

- CLI & logging (main.rs)
  - Parse flags (--input, -k, -m, -s, -o, -v) and configure env_logger.
- Data ingestion
  - Call load_records, then sample_records to enforce the 1000 record limit to ensure fast run time.
- Feature Construction
  - Instantiate FeatureBuilder::new, then vectorize to obtain a numeric matrix.
- Clustering
  - Use k means, receiving centroid vectors and per record assignments.
- Reporting

        - Use cluster_sizes to print counts per cluster. Optionally, call write_assignments_csv to export the raw assignments.
    - Interpretations
        - Reconstruct feature names via builder.numeric_fields(), .categorical_values(), and .tag_list(), then for each centroid, sort features by value and print the top five most defining features.

D. Tests

- **cargo test output**:

    running 3 tests

    test analytics::tests::test_cluster_sizes_counts ... ok

    test clustering::tests::basic_kmeans ... ok

    test features::tests::test_feature_dimension_and_vector ... ok

    test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

- Basic_kmeans (in clustering.rs):
    - Checks: Make sure two distinct 1-D points are each assigned to their own cluster when k=2.
    - Importance: Verifies that the core K-means loop converges correctly on a trivial case. This ensures assignment and centroid update steps are implemented without errors in logic or indexing.
- test_cluster_sizes_counts (in analytics.rs)
    - Checks: Given a synthetic assignments vector and a k value, the cluster_sizes returns the correct output.
    - Importance: confirms that cluster tallying logic correctly counts each level, which is critical for accurate reporting of how many records fell into each cluster.
- test_feature_dimension_and_vector (in features.rs)
    - Checks: dimension() reported by FeatureBuilder matches the length of the vector produced by vectorize(). Also checks a known numeric field (“age”: “30”) parses to 30.0 in the first element of the feature vector.
    - Importance: Ensures that feature‑extraction and vector‑length bookkeeping are in sync, and that numeric parsing works as intended.

E. Results

- **Program Output:**

K-means Clustering on Dating App Data

cluster sizes:

Cluster 0: 217 records

Cluster 1: 203 records

Cluster 2: 203 records

Cluster 3: 212 records

Cluster 4: 165 records

Cluster centroids (top 5 features):

Cluster 0:

- bio_length 418.442
- app_usage_time_min 215.631
- height_cm 173.770
- likes_received 100.074
- weight_kg 74.025

Cluster 1:

- bio_length 373.246
- height_cm 172.724
- likes_received 95.074
- weight_kg 72.101
- app_usage_time_min 62.532

Cluster 2:

- height_cm 170.488
- bio_length 103.315

- likes_received 101.970
- app_usage_time_min 75.655
- weight_kg 71.031

Cluster 3:

- bio_length 239.637
- app_usage_time_min 206.594
- height_cm 172.288
- likes_received 104.613
- weight_kg 70.309

Cluster 4:

- app_usage_time_min 223.715
- height_cm 170.576
- likes_received 100.418
- bio_length 74.455
- weight_kg 71.976

- Interpretation in project context (no need for "groundbreaking" results): In context of the project, each cluster represents a different demographic of people who share similar features. The first part of the output shows the number of records, or users, in each cluster. Each cluster themself represents a different demographic of the users of this synthetic dating app.
    - Cluster 0 has the longest bio length by far and the app usage time is also very high. The height is also above average with a solid amount of likes received. The interpretation from this is that these users are heavily invested in their profile, as seen by longer bios, and spend a great deal of time on this "app".
    - Cluster 1 had one of the longer bio lengths and is on the taller side compared to the average user and other clusters. The likes received are a bit lower than other clusters with also a low app usage and moderate weight. This shows that this demographic gives attention to details, especially their bios, but only checks the app occasionally.
    - Cluster 2 is our baseline demographic. Their height, usage time, and weight are all average. Their bio length is below average, around 100 characters, and their likes received is a little above average. This is a very middle of the road group, which concise

profile bios that still attract an average number of likes and a reasonable amount of time spent on the app.
    - Cluster 3 has the highest number of likes received and a very high app usage. Their bio-length is in the mid-range, with weight slightly below average. Their height is on the higher side, however. This group is the most engaged but also well-liked users. Their profiles are fairly detailed, they use the app heavily, and get the most likes.
    - Cluster 4 has the highest usage time, around 224 minutes. The height and likes received are both in the average range. The same is true for their weight. However, their bio length is the shortest. This group of people are the "browsers", they spend a lot of time on the app but their profiles are sparse.

F. Usage Instructions

- Build code:
    - Cargo build --release
- Run code
    - Cargo run --release

- Description of any command-line arguments or user interaction in the terminal
    - --input dating_app_behavior_dataset_extended1.csv: this provides the path to the input CSV file of user records
    - -- clusters 5: this is the number of clusters (k) to form in k-means (the hyperparameter)
    - -- max-iter 100: maximum number of k-means iterations before stopping
    - -- sample 1000: if the CSV file has more than 1000 rows, it will randomly sample 1000 rows. This is the maximum number of records to load and cluster.
    - -- output clusters.csv: optional path to write a CSV of "row_index, cluster". If this is omitted, no file is written.
    - -- verbose: enable info-level logging. Otherwise, only warnings and errors will show.
    - -- help: shows the help message and exit.
    - -- version: show program version and exist.

- Include expected runtime especially if your project takes a long time to run.
    - The project does not take long to run. It takes about 5-15 seconds.